REPORT 12 - CMM L1 to L2 Plan

Levi Nelson, Carlos W. Mercado

Brigham Young University-Idaho

CSE 272 - Software Lifecycle Models

Instructor: Br. Edward Bullen

Date: 03/29/2025

1. OVERVIEW

1.1 Project Introduction

> *The Capability Maturity Model (CMM) is a framework for assessing and improving an organization's process maturity, primarily in software development.*

The CMM assists software developers and management on how to control development and operations practices and create excellent products in well-managed organizations. Through focus on a small set of key process areas, developers can improve the software development process organization-wide (Paulk et al., 1993). Organizations that do not follow CMM guidelines are considered Level One organizations. However, organizations that follow the first set of basic CMM guidelines for proper software engineering and team management are considered Level Two teams. For this level, which "focus on establishing basic project-management controls" (Paulk et al., 1993a), the required key practices are:

A. *Requirements Management* - See section **4.2** for *Scrum* implementation.
- Means establishing a common understanding between the customer and a project team of the customer requirements. This agreement is the basis for planning and managing a project.

B. *Software Project Planning* - See section **2.2** for *Waterfall* implementation.
- Means establishing reasonable plans for engineering and managing a project. These plans are the foundation project management.

C. *Software Project Tracking and Oversight* - Omitted in this report.

- Means to establish adequate visibility into actual progress so that management can take effective action when a project's performance deviates significantly from the plans.

D. *Software Subcontract Management* - Omitted in this report.

- Means to select qualified subcontractors and manage them effectively.

E. *Software Quality Assurance* - See section **3.2** for *XP* implementation.

- Means to provide management with appropriate visibility into the process being used and the products being built.

F. *Software Configuration Management* - See section **5.2** for *DevOps* implementation.

- Means to establish and maintain the integrity of a project's products throughout its life cycle.

The omission of KPAs (C) and (D) were performed under the basis of selecting only the KPAs (A, B, E, F) that we considered were more suitable to the methodologies under analysis.

The purpose of this report is to discuss how the Waterfall, XP, Scrum, and DevOps software lifecycle methodologies (SLCM) can be affected by the implementation of the CMM methodology. For this end, and for the sake of briefly, we will pair each key practice to one SLCM at the time, omitting the complete implementation. This procedure will allow us to quickly gain a simple insight about the possibilities of implementing the whole CMM on the SLCM we've proposed for them. This way, we will help or hinder a Level One (the "initial level") team's mission to move to Level Two (the "repeatable level").

Hence, for each methodology, a portion of the methodology will be discussed and related to a specific CMM key process area. The relation of each methodology to the key process area will

be justified, and the methodology will be reviewed to determine if it helps or hinders the process of rising from Level One to Level Two.

Keywords: software development methodologies, CMM, Capability Maturity Model, Waterfall, XP, Extreme Programming, Agile, DevOps, DevSecOps, Operations, Development, management framework Scrum

2. WATERFALL ANALYSIS

2.1 Part of the Methodology

Name	Quote	Description
Preliminary and Final Program Design Documents	***Document No. 2 Preliminary Design*** (Spec) ***Document No. 4 Final Design*** (Spec) ***Document No. 4 Final Design*** (As Built) --- "If the total resources to be applied are insufficient or if the embryo operational design is wrong it will be recognized at this earlier stage and the iteration with requirements and preliminary design can be redone before final design, coding and test commences." (Royce 1970) --- "Write an overview document that is understandable, informative and current. Each and every worker must have an elemental understanding of the system. At least one person must have a deep understanding of the system which comes partially from having had to write an overview document." (Royce 1970)	Dr. Royce describes the importance of design documentation. He mentions the benefits of providing engineers with all necessary planning information. He also mentions that every worker must have an elemental understanding of the product, which is made possible through planning documentation.

2.2 Key Process Area

Name	Quote	Justification
(**B**) Software Project Planning	***Activity 2***: Software project planning is initiated in the early stages of, and in parallel with, the overall project planning. ***Activity 7***: The plan for the software project is documented. --- "The purpose of Software Project Planning is to establish reasonable plans for performing the software engineering and for managing the software project… Without realistic plans, effective project management cannot be implemented… Software Project Planning involves developing estimates for the work to be performed, establishing the necessary commitments, and defining the plan to do the work." (Paulk et al, 1993)	Paulk explains that a software project plan must be created and documented for effective software engineering and managing the product. Without these plans, product management will fail. This is related to the Preliminary and Final Design documents because they detail the plans to create a software project and the work to be performed.

2.3 Help/Hinder

Help/Hinder	Description
Help	Royce's design documents are created in one of the earliest stages of the methodology. They are created as a plan for the entire software project in

	parallel with other aspects of project planning (Royce 1970). They thoroughly document the plan for the software to be created. These documents establish plans for the engineering to be done, estimate the work to be performed, describe necessary commitments, and plan how to complete the work. Since these design documents fulfill many of the activities for Software Project Planning, they help rather than hinder.

2.4 References

- Royce, W. (1970), *Managing the Development of Large Software Systems, IEEE,* Available at: <https://byui-cse.github.io/cse272-course/Readings/Waterfall-Royce.pdf> (Accessed: 28 March 2025)

- Paulk, M. et al (1993), *Key Practices of the Capability Maturity Model(SM), Version 1.1, Software Engineering Institute of Carnegie Mellon University,* Available at: <https://byui-cse.github.io/cse272-course/Readings/CMM-Paulk-Complete.pdf> (Accessed: 28 March 2025)

3. XP ANALYSIS (Extreme Programming)

3.1 Part of the Methodology

Name	Quote	Description
XP Management Strategy	"The implementation of XP should be based on the *strategies* of the XP project management where almost all XP *values*, *principles* and *practices* are employed in order to deal with a particular strategy. For example, the *Management Strategy* that emerges from evaluation/use of *Accepted responsibility*, *Quality work*, *Incremental change*, *Local adoption*, *Travel light* and *Honest measurement* principles will guide us towards decentralised decision making and leave managers with Planning game practice, using metrics as the basic XP management tool." (Juric, 2000) "The project velocity (or just velocity) is a measure of how much work is getting done on your project. To measure the project velocity you simply add up the estimates of the user stories that were finished during the iteration. It's just that simple. You also total up the estimates for the tasks finished during the iteration. Both of these measurements are used for iteration planning." (Wells, 1999)	XP emphasizes speed delivery and small team work (and even one man work) over complex procedures and corporation size teams. Nevertheless, quality work and done-work measurement is estimated and used in planning.

3.2 Key Process Area

Name	Quote	Justification
(**E**) Software Quality Assurance	**Activity 3**: The SQA group participates in the preparation and review of the project's software development plan, standards, and procedures. **Activity 6**: The SQA group periodically reports the results of its activities to the software engineering group. --- "The purpose of Software Quality Assurance is to provide management with appropriate visibility into the process being used by the software project and of the products being built. Software Quality Assurance involves reviewing and auditing the software products and activities to verify that they comply with the applicable procedures and standards and providing the software project and other appropriate managers with the results of these reviews and audits. The software quality assurance group works with the software project during its early stages to establish plans, standards, and procedures that will add value to the software project and satisfy the constraints of the	CMM, regarding quality assurance, emphasizes the importance of good management and standardized quality control. This could potentially render useful XP Management Strategy, which focuses on correct measurement of the work done, under the expectation of that work being of good quality.

	project and the organization's policies. By participating in establishing the plans, standards, and procedures, the software quality assurance group helps ensure they fit the project's needs and verifies that they will be usable for performing reviews and audits throughout the software life cycle. The software quality assurance group reviews project activities and audits software work products throughout the life cycle and provides management with visibility as to whether the software project is adhering to its established plans, standards, and procedures." (Paulk et al, 1993).	

3.3 Help/Hinder

Help/Hinder	Description
Hinder	The *CMM Software Quality Assurance KPA*, regarding the presented activities (and also probably the other activities in the same KPA), may not be completely useful to the XP SLCM, due to its inherent "extreme nature". The CMM seems to be more focused in corporate level work improvement implementations, with big and standardized teams and practices, while the XP methodology seems to be more suitable for small teams (sometimes very small teams) working on smaller projects. Therefore, it is more of a hindrance than a help.

3.4 References

- Juric, R. (2000), *Extreme programming and its development practices*. ITI2000. Proceedings of the 22nd International Conference on Information Technology Interfaces (Cat. No.00EX411), Pula, Croatia, 2000, pp. 97-104. Available at:

<https://www.researchgate.net/publication/3893585_Extreme_programming_and_its_development_practices>. (Accessed: 29 March 2025)

- Paulk, M. et al (1993), *Key Practices of the Capability Maturity Model(SM), Version 1.1, Software Engineering Institute of Carnegie Mellon University,* Available at: <https://byui-cse.github.io/cse272-course/Readings/CMM-Paulk-Complete.pdf> (Accessed: 29 March 2025)
- Wells, D. (1999) *Extreme programming: A gentle introduction - Project Velocity*. Available at: <http://www.extremeprogramming.org/rules/velocity.html>. (Accessed: 29 March 2025).

4. SCRUM ANALYSIS

4.1 Part of the Methodology

Name	Quote	Description
Requirement Analysis	"... a team that needs additional expertise in product requirements will benefit from increased Product Owner involvement, including Daily Scrum attendance." "The Sprint Review Meeting is the appropriate meeting for external stakeholders (even end users) to attend. It is the opportunity to inspect and adapt the product as it emerges, and iteratively refine everyone's understanding of the requirements." "Scrum addresses uncertain **requirements** and technology risks by grouping people from multiple disciplines into one team (ideally in one team room) to maximize communication bandwidth, visibility, and trust. When **requirements** are uncertain and technology risks are high, adding too many people to the situation makes things worse. Grouping people by specialty also makes things worse. Grouping people by architectural components (a.k.a. component teams) makes things worse . . . eventually." (James, 2012)	Scrum requires detailed requirements, which are known by the Scrum Product Owner, final arbiter on requirements questions. The assistance of external stakeholders (and even end users) also helps when getting requirements straight. One key feature of requirements on the Scrum methodology, is their permanent state of adaptability.

4.2 Key Process Area

Name	Quote	Justification
(**A**) Requirements Management	**Activity 1**: The software engineering group reviews the allocated requirements before they are incorporated into the software project. --- "The purpose of Requirements Management is to establish a common understanding between the customer and the software project of the customer's requirements that will be addressed by the software project. Requirements Management involves establishing and maintaining an agreement with the customer on the requirements for the software project. This agreement is referred to as the "system requirements allocated to the software." The "customer" may be interpreted as the system engineering group, the marketing group, another internal organization, or an external customer. The agreement covers both the technical and nontechnical (e.g., delivery dates) requirements. The agreement forms the basis for estimating, planning, performing, and tracking the software project's activities throughout the software life cycle." (Paulk et al, 1993)	Paulk et al. explains that requirements are allocated beforehand into the project. The advantage of this proceeding is to allocate resources correctly and make any project progress orderly. This also allows management to make the necessary changes and avoid possible delays. Multiple teams may be involved in this activity (software engineering team, system test team, contract management team, etc.).

4.3 Help/Hinder

Help/Hinder	Description
Help	The use of an organized project requirements analysis and allocation, as proposed by the CCM, may help to mitigate any uncertainty or risk that may arise from the Scrum implementation. Requirement analysis requires review of requirements by the project manager and stakeholders. It helps to create understanding between the customer's requirements and the software product. Therefore, it is helpful rather than a hindrance.

4.4 References

- James, M. (2012) *Scrum reference card*. Available at: <https://scrumreferencecard.com/scrum-reference-card/> (Accessed: 29 March 2025)
- Paulk, M. et al (1993), *Key Practices of the Capability Maturity Model(SM), Version 1.1, Software Engineering Institute of Carnegie Mellon University,* Available at: <https://byui-cse.github.io/cse272-course/Readings/CMM-Paulk-Complete.pdf> (Accessed: 29 March 2025)

5. DevOps ANALYSIS

5.1 Part of the Methodology

Name	Quote	Description
Continuous Integration	"DevOps is defined as a practice that emphasizes the tasks enabling continuous integration between software development and its operational deployment needs." (Jabbari 2016) "[Continuous Integration is] the practice used by development teams to automate, merge, and test code. CI helps to catch bugs early in the development cycle, which makes them less expensive to fix. Automated tests execute as part of the CI process to ensure quality. CI systems produce artifacts and feed them to release processes to drive frequent deployments." (Nelson & Mercado, 2025).	Continuous integration is the process of automatically building, testing, and merging code into a shared repository. It is a central component of the DevOps process. Developers frequently commit to a version control system, which allows for early bug detection, higher software quality, and rapid feedback loops.

5.2 Key Process Area

Name	Quote	Justification
(**F**) Software Configuration Management	***Activity 8***: The software work products to be placed under configuration management are identified.	Paulk explains that all software work products, from documentation to compilers to software code units, should be placed under configuration management and identified. He also

	Examples of software work products that may be identified as configuration items/units include: ● Process-related documentation (e.g., plans, standards, or procedures) ● Software requirements ● Software design ● Software code units ● Software test procedures ● Software system build for the software test activity ● Software system build for delivery to the customer or end users ● Compilers ● Other support tools "The purpose of Software Configuration Management is to establish and maintain the integrity of the products of the software project throughout the project's software life cycle." "Software Configuration Management involves identifying the configuration of the software at given points in time, systematically controlling changes to the configuration, and maintaining the	explains that the purpose of software configuration management is to establish integrity of the product, and that the configuration management process involves controlling changes to the configuration and maintaining traceability of the software. It is related to continuous integration because both refer to constant management and repositories of software and artifacts.

	integrity and traceability of the configuration throughout the software life cycle." (Paulk et al, 1993).	

5.3 Help/Hinder

Help/Hinder	Description
Help	Continuous integration manages code versions, builds, and deployments. These items fulfill some of the suggestions given by Paulk as software work products that may be identified as configuration items (Paulk et al, 1993). Additionally, a shared repository and version control system allows for integrity of the product due to early bug detection and rapid feedback loops. Version control systems also maintain traceability of the software and its configuration. This continuous integration process is beneficial to the configuration management process, because continuous integration overlaps with several responsibilities. Therefore, continuous integration helps rather than hinders the configuration management process.

5.4 References

- Jabbari, R. *et al.* (2016) *What is DevOps?: A Systematic Mapping Study on Definitions and Practices*, *ResearchGate.net*. Available at: <https://www.researchgate.net/publication/308857081_What_is_DevOps_A_Systematic_Mapping_Study_on_Definitions_and_Practices> (Accessed: 28 March 2025).

- Juric, R. (2000), *Extreme programming and its development practices*. ITI2000. Proceedings of the 22nd International Conference on Information Technology Interfaces (Cat. No.00EX411), Pula, Croatia, 2000, pp. 97-104. Available at:

<https://www.researchgate.net/publication/3893585_Extreme_programming_and_its_development_practices>. (Accessed: 29 March 2025)

- Nelson L. & Mercado C. (2025) *W10 Report - DevOps Plan*. Available at: <https://docs.google.com/document/d/e/2PACX-1vSaDR8xbc_FGiw_YBXvtCsLZWh3AjO9eFcFdkNRg1BdDiuYVx6OnGQDW6TgIOaEUA/pub> (Accessed: 28 March 2025)

- Paulk, M et al (1993), *Key Practices of the Capability Maturity Model(SM), Version 1.1, Software Engineering Institute of Carnegie Mellon University,* Available at: <https://byui-cse.github.io/cse272-course/Readings/CMM-Paulk-Complete.pdf> (Accessed 28 March 2025).

6. REFERENCES

1. Jabbari, R. *et al.* (2016) *What is DevOps?: A Systematic Mapping Study on Definitions and Practices*, *ResearchGate.net*. Available at: <https://www.researchgate.net/publication/308857081_What_is_DevOps_A_Systematic_Mapping_Study_on_Definitions_and_Practices> (Accessed: 28 March 2025)

2. James, M. (2012) *Scrum reference card*. Available at: <https://scrumreferencecard.com/scrum-reference-card/> (Accessed: 29 March 2025)

3. Juric, R. (2000), *Extreme programming and its development practices*. ITI2000. Proceedings of the 22nd International Conference on Information Technology Interfaces (Cat. No.00EX411), Pula, Croatia, 2000, pp. 97-104. Available at: <https://www.researchgate.net/publication/3893585_Extreme_programming_and_its_development_practices>. (Accessed: 29 March 2025)

4. Nelson L. & Mercado C. (2025) *W10 Report - DevOps Plan*. Available at: <https://docs.google.com/document/d/e/2PACX-1vSaDR8xbc_FGiw_YBXvtCsLZWh3AjO9eFcFdkNRg1BdDiuYVx6OnGQDW6TgIOaEUA/pub> (Accessed: 28 March 2025)

5. Paulk, M. et al (1993), *Key Practices of the Capability Maturity Model(SM), Version 1.1, Software Engineering Institute of Carnegie Mellon University,* Available at: <https://byui-cse.github.io/cse272-course/Readings/CMM-Paulk-Complete.pdf> (Accessed 28 March 2025)

6. Paulk, M. *et al.* (1993a) *Capability maturity model, version 1.1*, *Capability maturity model, version 1.1 | IEEE Journals & Magazine | IEEE Xplore*. Available at: <https://ieeexplore.ieee.org/document/219617/> (Accessed: 29 March 2025)

7. Royce, W (1970), *Managing the Development of Large Software Systems, IEEE,* Available at: <https://byui-cse.github.io/cse272-course/Readings/Waterfall-Royce.pdf> (Accessed: 28 March 2025)

8. Wells, D. (1999) *Extreme programming: A gentle introduction - Project Velocity*. Available at: <http://www.extremeprogramming.org/rules/velocity.html>. (Accessed: 29 March 2025).